ITNE352-Project-Group-A4

Name	Student ID
layla khalil	202302358
Mohammed jamal	202302745

A Python socket-based client-server application that lets multiple users search and browse recipes from TheMealDB API in real time.

Table of Contents

Project Overview

The server connects to TheMealDB API and handles multiple clients simultaneously using threads. Each client gets an interactive menu to search, filter, and browse recipes. Every recipe request is saved as a JSON file on the server for testing and evaluation.

Requirements

- Python 3.8 or higher
- No external libraries needed — uses only Python standard library:
  - `socket`
  - `json`
  - `threading`
  - `urllib.request`

How to Run

1. Start the server

```
python server.py
```

The server will fetch reference data from TheMealDB and save it to `reference_A4.json`, then start listening on `127.0.0.1:12000`

2. Start the client (in a separate terminal)

```
python client.py
```

Enter your name when prompted. You can open multiple clients at the same time.

3. Navigate the menus

```
Main Menu
──────────────────────────────────────────────────
  1. Browse recipes
  2. Reference lists
  3. Quit
```

Features

Recipes Menu

- Search recipes by name (keyword search)
- Filter by category (e.g. Beef, Chicken, Dessert)
- Filter by area/cuisine (e.g. Italian, Japanese, Moroccan)
- Filter by ingredient
- Get a random recipe
- View full recipe detail (ingredients, instructions, YouTube link)

Reference Menu

- List all available categories
- List all available areas/cuisines
- List all available ingredients (first 50)

Project Structure

```
ITNE352-PROJECT-GROUP-A4/
│
├── server.py                    # Server — handles all clients and API calls
├── client.py                    # Client — interactive menu interface
└── README.md
```

OOP Design

The project is fully structured using Object-Oriented Programming. Each class has a single, clear responsibility.

Server Classes

Class	Responsibility
`MessageHelper`	Handles the length-prefixed JSON send/receive protocol
`MealDBClient`	Makes all HTTP requests to TheMealDB API
`RecipeFormatter`	Transforms raw API data into brief lists or full detail
`ReferenceCache`	Loads and stores categories, areas, and ingredients at startup
`ClientHandler`	Manages one client's full request/response lifecycle
`RecipeServer`	Owns the server socket, accepts connections, spawns threads

Client Classes

Class	Responsibility
`MessageHelper`	Same protocol framing as server side
`Display`	All terminal output (recipe lists, details, headers)
`InputHelper`	All validated user input (menus, prompts)
`BaseMenu`	Abstract base — shared socket, display, and input helpers
`RecipesMenu`	Inherits `BaseMenu` — handles recipe search/filter screens
`ReferenceMenu`	Inherits `BaseMenu` — handles reference list screens
`MainMenu`	Inherits `BaseMenu` — top-level navigation
`RecipeClient`	Manages connection lifecycle and starts the application

OOP Concepts Used

- **Encapsulation** — Internal attributes and methods use `_` prefix; only public interfaces are exposed
- **Inheritance** — `RecipesMenu`, `ReferenceMenu`, and `MainMenu` all inherit shared functionality from `BaseMenu`
- **Polymorphism** — `MainMenu` calls `.run()` on any menu object through the same interface
- **Abstraction** — `BaseMenu.run()` defines a contract that all subclasses must implement